

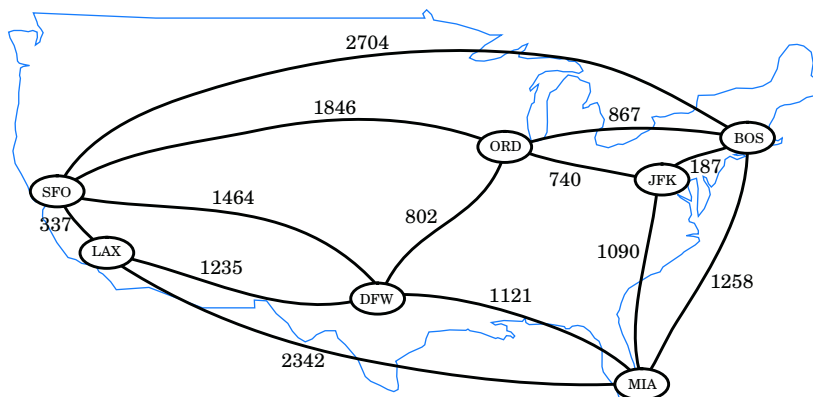
## 14.6 Shortest Paths

As we saw in Section 14.3.3, the breadth-first search strategy can be used to find a path with as few edges as possible from some starting vertex to every other vertex in a connected graph. This approach makes sense in cases where each edge is as good as any other, but there are many situations where this approach is not appropriate.

For example, we might want to use a graph to represent the roads between cities, and we might be interested in finding the fastest way to travel cross-country. In this case, it is probably not appropriate for all the edges to be equal to each other, for some inter-city distances will likely be much larger than others. Likewise, we might be using a graph to represent a computer network (such as the Internet), and we might be interested in finding the fastest way to route a data packet between two computers. In this case, it again may not be appropriate for all the edges to be equal to each other, for some connections in a computer network are typically much faster than others (for example, some edges might represent low-bandwidth connections, while others might represent high-speed, fiber-optic connections). It is natural, therefore, to consider graphs whose edges are not weighted equally.

### 14.6.1 Weighted Graphs

A **weighted graph** is a graph that has a numeric (for example, integer) label  $w(e)$  associated with each edge  $e$ , called the **weight** of edge  $e$ . For  $e = (u, v)$ , we let notation  $w(u, v) = w(e)$ . We show an example of a weighted graph in Figure 14.14.



**Figure 14.14:** A weighted graph whose vertices represent major U.S. airports and whose edge weights represent distances in miles. This graph has a path from JFK to LAX of total weight 2,777 (going through ORD and DFW). This is the minimum-weight path in the graph from JFK to LAX.

### Defining Shortest Paths in a Weighted Graph

Let  $G$  be a weighted graph. The **length** (or weight) of a path is the sum of the weights of the edges of  $P$ . That is, if  $P = ((v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k))$ , then the length of  $P$ , denoted  $w(P)$ , is defined as

$$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}).$$

The **distance** from a vertex  $u$  to a vertex  $v$  in  $G$ , denoted  $d(u, v)$ , is the length of a minimum-length path (also called **shortest path**) from  $u$  to  $v$ , if such a path exists.

People often use the convention that  $d(u, v) = \infty$  if there is no path at all from  $u$  to  $v$  in  $G$ . Even if there is a path from  $u$  to  $v$  in  $G$ , however, if there is a cycle in  $G$  whose total weight is negative, the distance from  $u$  to  $v$  may not be defined. For example, suppose vertices in  $G$  represent cities, and the weights of edges in  $G$  represent how much money it costs to go from one city to another. If someone were willing to actually pay us to go from say JFK to ORD, then the “cost” of the edge (JFK, ORD) would be negative. If someone else were willing to pay us to go from ORD to JFK, then there would be a negative-weight cycle in  $G$  and distances would no longer be defined. That is, anyone could now build a path (with cycles) in  $G$  from any city  $A$  to another city  $B$  that first goes to JFK and then cycles as many times as he or she likes from JFK to ORD and back, before going on to  $B$ . The existence of such paths would allow us to build arbitrarily low negative-cost paths (and, in this case, make a fortune in the process). But distances cannot be arbitrarily low negative numbers. Thus, any time we use edge weights to represent distances, we must be careful not to introduce any negative-weight cycles.

Suppose we are given a weighted graph  $G$ , and we are asked to find a shortest path from some vertex  $s$  to each other vertex in  $G$ , viewing the weights on the edges as distances. In this section, we explore efficient ways of finding all such shortest paths, if they exist. The first algorithm we discuss is for the simple, yet common, case when all the edge weights in  $G$  are nonnegative (that is,  $w(e) \geq 0$  for each edge  $e$  of  $G$ ); hence, we know in advance that there are no negative-weight cycles in  $G$ . Recall that the special case of computing a shortest path when all weights are equal to one was solved with the BFS traversal algorithm presented in Section 14.3.3.

There is an interesting approach for solving this **single-source** problem based on the **greedy-method** design pattern (Section 13.4.2). Recall that in this pattern we solve the problem at hand by repeatedly selecting the best choice from among those available in each iteration. This paradigm can often be used in situations where we are trying to optimize some cost function over a collection of objects. We can add objects to our collection, one at a time, always picking the next one that optimizes the function from among those yet to be chosen.

### 14.6.2 Dijkstra's Algorithm

The main idea in applying the greedy-method pattern to the single-source shortest-path problem is to perform a “weighted” breadth-first search starting at the source vertex  $s$ . In particular, we can use the greedy method to develop an algorithm that iteratively grows a “cloud” of vertices out of  $s$ , with the vertices entering the cloud in order of their distances from  $s$ . Thus, in each iteration, the next vertex chosen is the vertex outside the cloud that is closest to  $s$ . The algorithm terminates when no more vertices are outside the cloud (or when those outside the cloud are not connected to those within the cloud), at which point we have a shortest path from  $s$  to every vertex of  $G$  that is reachable from  $s$ . This approach is a simple, but nevertheless powerful, example of the greedy-method design pattern. Applying the greedy method to the single-source, shortest-path problem, results in an algorithm known as *Dijkstra's algorithm*.

#### Edge Relaxation

Let us define a label  $D[v]$  for each vertex  $v$  in  $V$ , which we use to approximate the distance in  $G$  from  $s$  to  $v$ . The meaning of these labels is that  $D[v]$  will always store the length of the best path we have found *so far* from  $s$  to  $v$ . Initially,  $D[s] = 0$  and  $D[v] = \infty$  for each  $v \neq s$ , and we define the set  $C$ , which is our “cloud” of vertices, to initially be the empty set. At each iteration of the algorithm, we select a vertex  $u$  not in  $C$  with smallest  $D[u]$  label, and we pull  $u$  into  $C$ . (In general, we will use a priority queue to select among the vertices outside the cloud.) In the very first iteration we will, of course, pull  $s$  into  $C$ . Once a new vertex  $u$  is pulled into  $C$ , we update the label  $D[v]$  of each vertex  $v$  that is adjacent to  $u$  and is outside of  $C$ , to reflect the fact that there may be a new and better way to get to  $v$  via  $u$ . This update operation is known as a *relaxation* procedure, for it takes an old estimate and checks if it can be improved to get closer to its true value. The specific edge relaxation operation is as follows:

#### Edge Relaxation:

```
if  $D[u] + w(u, v) < D[v]$  then  
     $D[v] = D[u] + w(u, v)$ 
```

#### Algorithm Description and Example

We give the pseudocode for Dijkstra's algorithm in Code Fragment 14.12, and illustrate several iterations of Dijkstra's algorithm in Figures 14.15 through 14.17.

**Algorithm** ShortestPath( $G, s$ ):

**Input:** A directed or undirected graph  $G$  with nonnegative edge weights, and a distinguished vertex  $s$  of  $G$ .

**Output:** The length of a shortest path from  $s$  to  $v$  for each vertex  $v$  of  $G$ .

Initialize  $D[s] = 0$  and  $D[v] = \infty$  for each vertex  $v \neq s$ .

Let a priority queue  $Q$  contain all the vertices of  $G$  using the  $D$  labels as keys.

**while**  $Q$  is not empty **do**

    {pull a new vertex  $u$  into the cloud}

$u =$  value returned by  $Q.\text{removeMin}()$

**for** each edge  $(u, v)$  such that  $v$  is in  $Q$  **do**

        {perform the *relaxation* procedure on edge  $(u, v)$ }

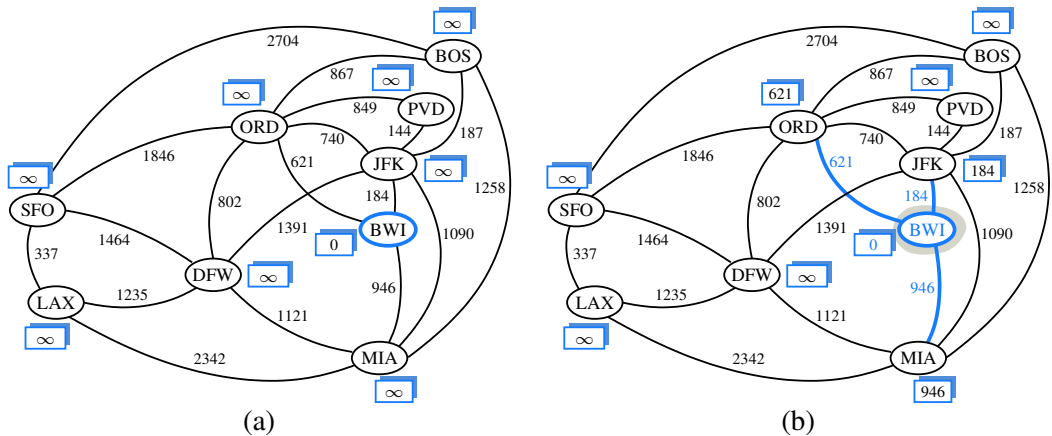
**if**  $D[u] + w(u, v) < D[v]$  **then**

$D[v] = D[u] + w(u, v)$

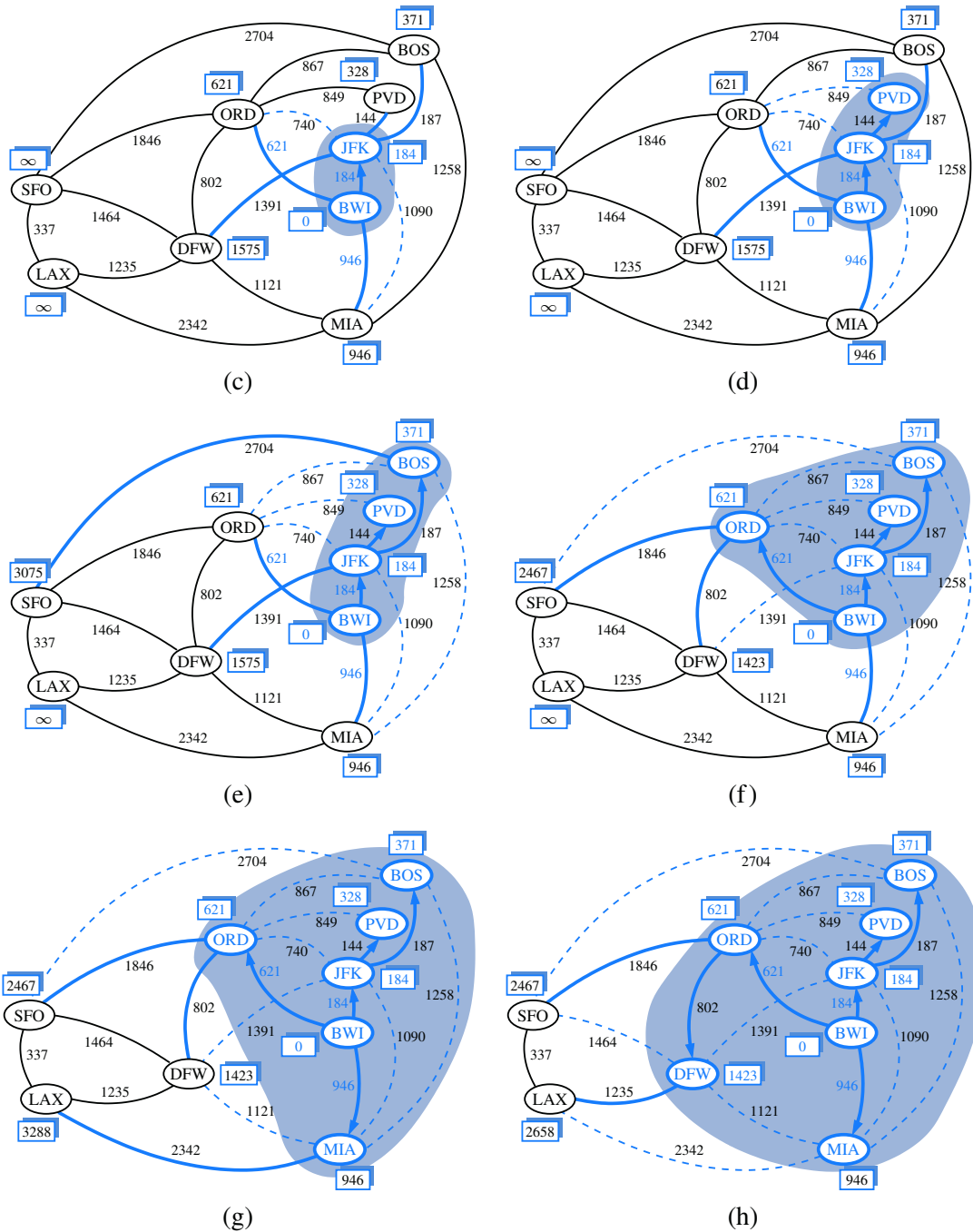
            Change the key of vertex  $v$  in  $Q$  to  $D[v]$ .

**return** the label  $D[v]$  of each vertex  $v$

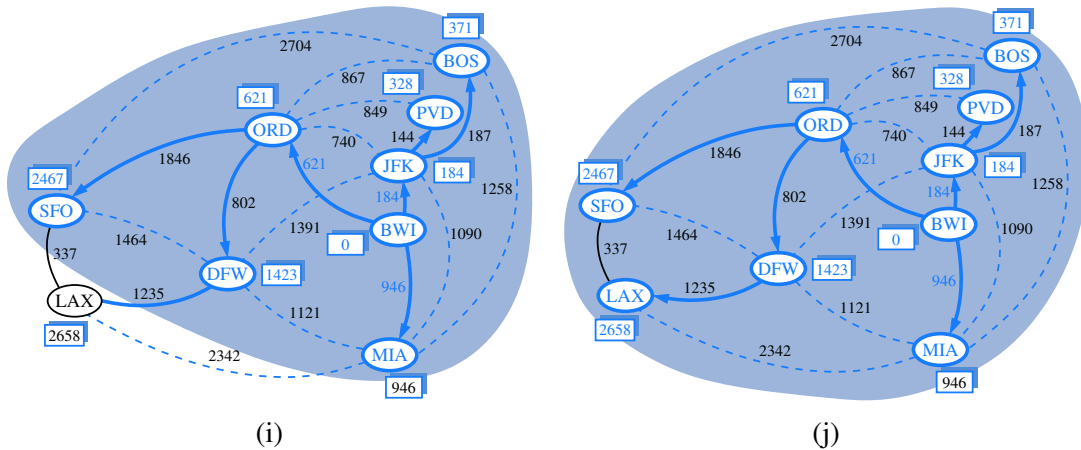
**Code Fragment 14.12:** Pseudocode for Dijkstra’s algorithm, solving the single-source shortest-path problem for an undirected or directed graph.



**Figure 14.15:** An example execution of Dijkstra’s shortest-path algorithm on a weighted graph. The start vertex is BWI. A box next to each vertex  $v$  stores the label  $D[v]$ . The edges of the shortest-path tree are drawn as thick arrows, and for each vertex  $u$  outside the “cloud” we show the current best edge for pulling in  $u$  with a thick line. (Continues in Figure 14.16.)



**Figure 14.16:** An example execution of Dijkstra's shortest-path algorithm on a weighted graph. (Continued from Figure 14.15; continues in Figure 14.17.)



**Figure 14.17:** An example execution of Dijkstra's shortest-path algorithm on a weighted graph. (Continued from Figure 14.16.)

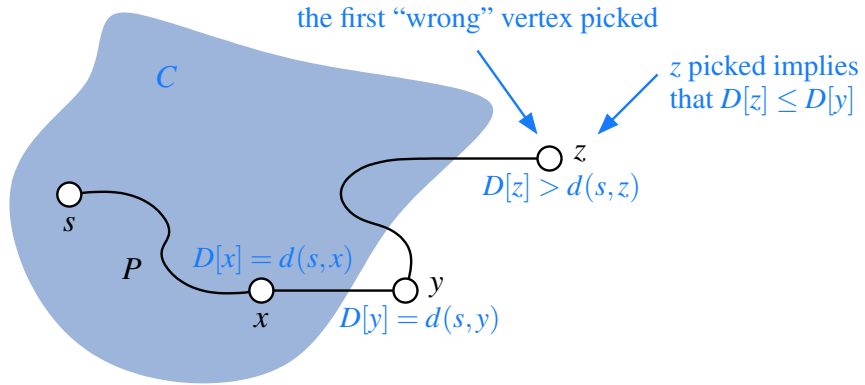
### Why It Works

The interesting aspect of the Dijkstra algorithm is that, at the moment a vertex  $u$  is pulled into  $C$ , its label  $D[u]$  stores the correct length of a shortest path from  $v$  to  $u$ . Thus, when the algorithm terminates, it will have computed the shortest-path distance from  $s$  to every vertex of  $G$ . That is, it will have solved the single-source shortest-path problem.

It is probably not immediately clear why Dijkstra's algorithm correctly finds the shortest path from the start vertex  $s$  to each other vertex  $u$  in the graph. Why is it that the distance from  $s$  to  $u$  is equal to the value of the label  $D[u]$  at the time vertex  $u$  is removed from the priority queue  $Q$  and added to the cloud  $C$ ? The answer to this question depends on there being no negative-weight edges in the graph, for it allows the greedy method to work correctly, as we show in the proposition that follows.

**Proposition 14.23:** *In Dijkstra's algorithm, whenever a vertex  $v$  is pulled into the cloud, the label  $D[v]$  is equal to  $d(s, v)$ , the length of a shortest path from  $s$  to  $v$ .*

**Justification:** Suppose that  $D[v] > d(s, v)$  for some vertex  $v$  in  $V$ , and let  $z$  be the *first* vertex the algorithm pulled into the cloud  $C$  (that is, removed from  $Q$ ) such that  $D[z] > d(s, z)$ . There is a shortest path  $P$  from  $s$  to  $z$  (for otherwise  $d(s, z) = \infty = D[z]$ ). Let us therefore consider the moment when  $z$  is pulled into  $C$ , and let  $y$  be the first vertex of  $P$  (when going from  $s$  to  $z$ ) that is not in  $C$  at this moment. Let  $x$  be the predecessor of  $y$  in path  $P$  (note that we could have  $x = s$ ). (See Figure 14.18.) We know, by our choice of  $y$ , that  $x$  is already in  $C$  at this point.



**Figure 14.18:** A schematic illustration for the justification of Proposition 14.23.

Moreover,  $D[x] = d(s, x)$ , since  $z$  is the **first** incorrect vertex. When  $x$  was pulled into  $C$ , we tested (and possibly updated)  $D[y]$  so that we had at that point

$$D[y] \leq D[x] + w(x, y) = d(s, x) + w(x, y).$$

But since  $y$  is the next vertex on the shortest path from  $s$  to  $z$ , this implies that

$$D[y] = d(s, y).$$

But we are now at the moment when we are picking  $z$ , not  $y$ , to join  $C$ ; hence,

$$D[z] \leq D[y].$$

It should be clear that a subpath of a shortest path is itself a shortest path. Hence, since  $y$  is on the shortest path from  $s$  to  $z$ ,

$$d(s, y) + d(y, z) = d(s, z).$$

Moreover,  $d(y, z) \geq 0$  because there are no negative-weight edges. Therefore,

$$D[z] \leq D[y] = d(s, y) \leq d(s, y) + d(y, z) = d(s, z).$$

But this contradicts the definition of  $z$ ; hence, there can be no such vertex  $z$ . ■

### The Running Time of Dijkstra's Algorithm

In this section, we analyze the time complexity of Dijkstra's algorithm. We denote with  $n$  and  $m$  the number of vertices and edges of the input graph  $G$ , respectively. We assume that the edge weights can be added and compared in constant time. Because of the high level of the description we gave for Dijkstra's algorithm in Code Fragment 14.12, analyzing its running time requires that we give more details on its implementation. Specifically, we should indicate the data structures used and how they are implemented.

Let us first assume that we are representing the graph  $G$  using an adjacency list or adjacency map structure. This data structure allows us to step through the vertices adjacent to  $u$  during the relaxation step in time proportional to their number. Therefore, the time spent in the management of the nested **for** loop, and the number of iterations of that loop, is

$$\sum_{u \text{ in } V_G} \text{outdeg}(u),$$

which is  $O(m)$  by Proposition 14.9. The outer **while** loop executes  $O(n)$  times, since a new vertex is added to the cloud during each iteration. This still does not settle all the details for the algorithm analysis, however, for we must say more about how to implement the other principal data structure in the algorithm—the priority queue  $Q$ .

Referring back to Code Fragment 14.12 in search of priority queue operations, we find that  $n$  vertices are originally inserted into the priority queue; since these are the only insertions, the maximum size of the queue is  $n$ . In each of  $n$  iterations of the **while** loop, a call to `removeMin` is made to extract the vertex  $u$  with smallest  $D$  label from  $Q$ . Then, for each neighbor  $v$  of  $u$ , we perform an edge relaxation, and may potentially update the key of  $v$  in the queue. Thus, we actually need an implementation of an *adaptable priority queue* (Section 9.5), in which case the key of a vertex  $v$  is changed using the method `replaceKey(e, k)`, where  $e$  is the priority queue entry associated with vertex  $v$ . In the worst case, there could be one such update for each edge of the graph. Overall, the running time of Dijkstra's algorithm is bounded by the sum of the following:

- $n$  insertions into  $Q$ .
- $n$  calls to the `removeMin` method on  $Q$ .
- $m$  calls to the `replaceKey` method on  $Q$ .

If  $Q$  is an adaptable priority queue implemented as a heap, then each of the above operations run in  $O(\log n)$ , and so the overall running time for Dijkstra's algorithm is  $O((n + m) \log n)$ . Note that if we wish to express the running time as a function of  $n$  only, then it is  $O(n^2 \log n)$  in the worst case.

Let us now consider an alternative implementation for the adaptable priority queue  $Q$  using an unsorted sequence. (See Exercise P-9.52.) This, of course, requires that we spend  $O(n)$  time to extract the minimum element, but it affords very fast key updates, provided  $Q$  supports location-aware entries (Section 9.5.1). Specifically, we can implement each key update done in a relaxation step in  $O(1)$  time—we simply change the key value once we locate the entry in  $Q$  to update. Hence, this implementation results in a running time that is  $O(n^2 + m)$ , which can be simplified to  $O(n^2)$  since  $G$  is simple.

### Comparing the Two Implementations

We have two choices for implementing the adaptable priority queue with location-aware entries in Dijkstra's algorithm: a heap implementation, which yields a running time of  $O((n + m) \log n)$ , and an unsorted sequence implementation, which yields a running time of  $O(n^2)$ . Since both implementations would be fairly simple to code, they are about equal in terms of the programming sophistication needed. These two implementations are also about equal in terms of the constant factors in their worst-case running times. Looking only at these worst-case times, we prefer the heap implementation when the number of edges in the graph is small (that is, when  $m < n^2 / \log n$ ), and we prefer the sequence implementation when the number of edges is large (that is, when  $m > n^2 / \log n$ ).

**Proposition 14.24:** *Given a weighted graph  $G$  with  $n$  vertices and  $m$  edges, such that the weight of each edge is nonnegative, and a vertex  $s$  of  $G$ , Dijkstra's algorithm can compute the distance from  $s$  to all other vertices of  $G$  in the better of  $O(n^2)$  or  $O((n + m) \log n)$  time.*

We note that an advanced priority queue implementation, known as a **Fibonacci heap**, can be used to implement Dijkstra's algorithm in  $O(m + n \log n)$  time.

### Programming Dijkstra's Algorithm in Java

Having given a pseudocode description of Dijkstra's algorithm, let us now present Java code for performing Dijkstra's algorithm, assuming we are given a graph whose edge elements are nonnegative integer weights. Our implementation of the algorithm is in the form of a method, `shortestPathLengths`, that takes a graph and a designated source vertex as parameters. (See Code Fragment 14.13.) It returns a map, named `cloud`, storing the shortest-path distance  $d(s, v)$  for each vertex  $v$  that is reachable from the source. We rely on our `HeapAdaptablePriorityQueue` developed in Section 9.5.2 as an adaptable priority queue.

As we have done with other algorithms in this chapter, we rely on hash-based maps to store auxiliary data (in this case, mapping  $v$  to its distance bound  $D[v]$  and its adaptable priority queue entry). The expected  $O(1)$ -time access to elements of these dictionaries could be converted to worst-case bounds, either by numbering vertices from 0 to  $n - 1$  to use as indices into an array, or by storing the information within each vertex's element.

The pseudocode for Dijkstra's algorithm begins by assigning  $D[v] = \infty$  for each  $v$  other than the source; we rely on the special value `Integer.MAX_VALUE` in Java to provide a sufficient numeric value to model infinity. However, we avoid including vertices with this "infinite" distance in the resulting `cloud` that is returned by the method. The use of this numeric limit could be avoided altogether by waiting to add a vertex to the priority queue until after an edge that reaches it is relaxed. (See Exercise C-14.62.)

```

1  /** Computes shortest-path distances from src vertex to all reachable vertices of g. */
2  public static <V> Map<Vertex<V>, Integer>
3  shortestPathLengths(Graph<V,Integer> g, Vertex<V> src) {
4      // d.get(v) is upper bound on distance from src to v
5      Map<Vertex<V>, Integer> d = new ProbeHashMap<>();
6      // map reachable v to its d value
7      Map<Vertex<V>, Integer> cloud = new ProbeHashMap<>();
8      // pq will have vertices as elements, with d.get(v) as key
9      AdaptablePriorityQueue<Integer, Vertex<V>> pq;
10     pq = new HeapAdaptablePriorityQueue<>();
11     // maps from vertex to its pq locator
12     Map<Vertex<V>, Entry<Integer,Vertex<V>>> pqTokens;
13     pqTokens = new ProbeHashMap<>();
14
15     // for each vertex v of the graph, add an entry to the priority queue, with
16     // the source having distance 0 and all others having infinite distance
17     for (Vertex<V> v : g.vertices()) {
18         if (v == src)
19             d.put(v,0);
20         else
21             d.put(v, Integer.MAX_VALUE);
22         pqTokens.put(v, pq.insert(d.get(v), v)); // save entry for future updates
23     }
24     // now begin adding reachable vertices to the cloud
25     while (!pq.isEmpty()) {
26         Entry<Integer, Vertex<V>> entry = pq.removeMin();
27         int key = entry.getKey();
28         Vertex<V> u = entry.getValue();
29         cloud.put(u, key); // this is actual distance to u
30         pqTokens.remove(u); // u is no longer in pq
31         for (Edge<Integer> e : g.outgoingEdges(u)) {
32             Vertex<V> v = g.opposite(u,e);
33             if (cloud.get(v) == null) {
34                 // perform relaxation step on edge (u,v)
35                 int wgt = e.getElement();
36                 if (d.get(u) + wgt < d.get(v)) { // better path to v?
37                     d.put(v, d.get(u) + wgt); // update the distance
38                     pq.replaceKey(pqTokens.get(v), d.get(v)); // update the pq entry
39                 }
40             }
41         }
42     }
43     return cloud; // this only includes reachable vertices
44 }

```

**Code Fragment 14.13:** Java implementation of Dijkstra’s algorithm for computing the shortest-path distances from a single source. We assume that *e*.getElement() for edge *e* represents the weight of that edge.

## Reconstructing a Shortest-Path Tree

Our pseudocode description of Dijkstra's algorithm in Code Fragment 14.12 and our implementation in Code Fragment 14.13 compute the value  $D[v]$ , for each vertex  $v$ , that is the length of a shortest path from the source vertex  $s$  to  $v$ . However, those forms of the algorithm do not explicitly compute the actual paths that achieve those distances. Fortunately, it is possible to represent shortest paths from source  $s$  to every reachable vertex in a graph using a compact data structure known as a **shortest-path tree**. This is possible because if a shortest path from  $s$  to  $v$  passes through an intermediate vertex  $u$ , it must begin with a shortest path from  $s$  to  $u$ .

We next demonstrate that a shortest-path tree rooted at source  $s$  can be reconstructed in  $O(n + m)$  time, given the  $D[v]$  values produced by Dijkstra's algorithm using  $s$  as the source. As we did when representing the DFS and BFS trees, we will map each vertex  $v \neq s$  to a parent  $u$  (possibly,  $u = s$ ), such that  $u$  is the vertex immediately before  $v$  on a shortest path from  $s$  to  $v$ . If  $u$  is the vertex just before  $v$  on a shortest path from  $s$  to  $v$ , it must be that

$$D[u] + w(u, v) = D[v].$$

Conversely, if the above equation is satisfied, then a shortest path from  $s$  to  $u$  followed by the edge  $(u, v)$  is a shortest path to  $v$ .

Our implementation in Code Fragment 14.14 reconstructs a tree based on this logic, testing all *incoming* edges to each vertex  $v$ , looking for a  $(u, v)$  that satisfies the key equation. The running time is  $O(n + m)$ , as we consider each vertex and all incoming edges to those vertices. (See Proposition 14.9.)

```

1  /**
2   * Reconstructs a shortest-path tree rooted at vertex s, given distance map d.
3   * The tree is represented as a map from each reachable vertex v (other than s)
4   * to the edge e = (u,v) that is used to reach v from its parent u in the tree.
5   */
6  public static <V> Map<Vertex<V>, Edge<Integer>>
7  spTree(Graph<V,Integer> g, Vertex<V> s, Map<Vertex<V>, Integer> d) {
8      Map<Vertex<V>, Edge<Integer>> tree = new ProbeHashMap<>();
9      for (Vertex<V> v : d.keySet())
10         if (v != s)
11             for (Edge<Integer> e : g.incomingEdges(v)) { // consider INCOMING edges
12                 Vertex<V> u = g.opposite(v, e);
13                 int wgt = e.getElement();
14                 if (d.get(v) == d.get(u) + wgt)
15                     tree.put(v, e); // edge is used to reach v
16             }
17         return tree;
18     }

```

**Code Fragment 14.14:** Java method that reconstructs a single-source shortest-path tree, based on knowledge of the shortest-path distances.